

Punto 6: Reflexión sobre el trabajo realizado

Tabla de apoyo para 6.a

Mapa	Tamaño	DFS costo/expansión	BFS costo/expansión	UCS costo/expansión
tinyDiagnostic	21×13	95 / 66	24 / 147	24 / 134
radiationShortcut	27×17	172 / 180	84 / 244	51 / 251
antennaServiceRing	37×21	326 / 390	86 / 410	59 / 413
orbitalDiagnosticComplex	43×25	196 / 192	101 / 591	62 / 560

6a. Análisis de los algoritmos y heurísticas

6.a.1.

DFS tiene complejidad temporal $O(b^m)$ y espacial $O(b \cdot m)$, ya que solo mantiene en memoria la rama actual. BFS es $O(b^d)$ tanto en tiempo como en espacio, porque almacena la frontera completa. UCS es $O(b^{(1+L \cdot C^*/\epsilon)})$, donde C^* es el costo óptimo y ϵ el costo mínimo de una acción. A^* se comporta como UCS cuando $h \equiv 0$, lo cual verificamos experimentalmente: A^* con nullHeuristic sobre radiationShortcut produjo un costo de 51 y 251 nodos expandidos, exactamente los mismos valores que UCS.

Para la medición empírica utilizamos nodos expandidos como proxy del tiempo, métrica estándar en AIMA, porque el cronómetro del proyecto reporta 0.0 s en todos los mapas de diagnóstico y no permite discriminar entre algoritmos. El tiempo solo se vuelve medible en el punto 5, donde el estado compuesto (posición, hasKit, pendingSystems) hace explotar el espacio de búsqueda: A^* con nullHeuristic expandió

16.147 nodos en 0.1 s, dos órdenes de magnitud por encima de los mapas de diagnóstico.

Los datos muestran que DFS expande consistentemente menos nodos que BFS y UCS en los cuatro mapas evaluados (66 frente a 147, 180 frente a 244, 390 frente a 410, y 192 frente a 591), porque se detiene con la primera meta que encuentra sin explorar alternativas. BFS y UCS están obligados a agotar una región completa antes de poder terminar: BFS todos los nodos hasta el nivel d , y UCS todos los estados con $g < C^*$, lo que explica su mayor número de expansiones. Este menor trabajo de DFS no constituye una ventaja, sino la contrapartida de no ofrecer ninguna garantía, y se paga en el costo de la solución (95 frente a 24 en tinyDiagnostic, y 326 frente a 59 en antennaServiceRing).

Finalmente, las expansiones de BFS y UCS crecen de forma aproximadamente proporcional al tamaño del mapa (147, 244, 410 y 591) y no exponencialmente como sugieren las cotas $O(b^d)$. Esto se explica porque implementamos búsqueda en grafo: el conjunto de estados alcanzados impide que un mismo estado se expanda más de una vez, colapsando el árbol de búsqueda sobre el grafo de espacio de estados. Las cotas

exponenciales describen el árbol, donde un mismo estado puede repetirse en ramas distintas. DFS, en cambio, no exhibe un crecimiento monótono: expandió 390 nodos en `antennaServiceRing` (37x21) pero solo 192 en `orbitalDiagnosticComplex` (43x25), pese a ser un mapa más grande, porque su desempeño depende enteramente de la rama que elija explorar primero.

6.a.2.

En los problemas de este taller los cuatro algoritmos son completos, pero por razones distintas y bajo condiciones que conviene precisar. Todos los mapas comparten dos propiedades que sustentan estas garantías: el factor de ramificación es finito, ya que el robot dispone de a lo sumo cuatro acciones (arriba, abajo, izquierda y derecha), y el espacio de estados es finito, porque la cuadrícula es acotada y las celdas marcadas como muro o compuerta bloqueada no son transitables.

BFS es completo siempre que el factor de ramificación sea finito, condición que se cumple en todos los mapas. Explora por niveles, de modo que si existe una solución a profundidad d , la encontrará después de agotar los niveles anteriores. El conjunto de estados alcanzados no es necesario para su completitud en un espacio finito, pero evita que un mismo estado ingrese varias veces a la frontera y reduce el trabajo total.

DFS es el caso que requiere mayor justificación, porque en general no es completo: en presencia de ciclos o de ramas de profundidad infinita puede explorar indefinidamente sin alcanzar la meta. En este taller sí lo es, y ello se debe a dos condiciones combinadas: la finitud del espacio de estados y el control de repetidos mediante el conjunto de estados alcanzados que incorporamos en la implementación. Lo determinante no es el tamaño del mapa sino esta combinación, ya que un mapa pequeño con ciclos y sin control de repetidos tampoco ofrecería la garantía. El mapa `antennaServiceRing` constituye la evidencia empírica de este punto: su estructura en anillo contiene ciclos y, aun así, DFS terminó y devolvió una solución.

UCS es completo siempre que todos los costos de acción sean mayores o iguales a un valor ϵ estrictamente positivo. La razón es que UCS expande siempre el nodo de menor costo acumulado g , de manera que su terminación depende de que g crezca a medida que se avanza. Si existieran acciones de costo cero, el algoritmo podría permanecer indefinidamente extrayendo nodos de un ciclo gratuito, puesto que siempre serían los más baratos de la cola, y nunca progresaría hacia la meta. Con la condición $\epsilon > 0$ se garantiza que tras k pasos el costo acumulado es al menos $k \cdot \epsilon$, cantidad que crece sin cota, por lo que todo camino suficientemente largo termina superando el costo óptimo C^* y deja de considerarse. En este taller la condición se cumple holgadamente, ya que los costos de terreno son 1, 2, 3 y 5, con lo cual $\epsilon = 1$. Los muros tienen costo infinito, pero esto no compromete la garantía porque nunca se generan como sucesores y, en consecuencia, no

forman parte del espacio de estados alcanzable.

A* hereda la completitud de UCS cuando la heurística es admisible y toma valores finitos. Su prioridad es $f = g + h$, y dado que g crece sin cota y h está acotada, f también crece, lo que preserva el mismo argumento de terminación. Con `nullHeuristic` el algoritmo coincide exactamente con UCS, hecho que verificamos experimentalmente al obtener resultados idénticos en costo y en nodos expandidos.

Conviene añadir una precisión sobre los puntos 4 y 5, donde el estado deja de ser únicamente la posición. En `ModuleRepairProblem` el espacio de estados tiene tamaño igual al número de celdas transitables multiplicado por dos, correspondiente a las dos configuraciones de `hasModule`, y en `SystemRepairProblem` es el número de celdas multiplicado por dos y por 2 elevado al número de sistemas averiados. Ambos siguen siendo finitos, de modo que las garantías de completitud se mantienen. No obstante, completitud y tratabilidad son propiedades distintas: la garantía teórica supone recursos ilimitados, mientras que en la práctica el límite lo imponen la memoria y el tiempo disponibles. El enunciado advierte precisamente que algunas instancias de layouts/systems pueden tener un espacio de búsqueda tan grande que no se resuelvan en un tiempo razonable, lo cual no contradice la completitud del algoritmo sino que refleja el crecimiento combinatorio del espacio de estados.

Empíricamente, las trece ejecuciones realizadas sobre los cuatro mapas de diagnóstico devolvieron una solución en todos los casos, sin fallos ni ejecuciones que no terminaran, lo cual es consistente con las garantías descritas.

6.a.3

DFS no es óptimo bajo ninguna condición. Retorna el primer camino que encuentra hasta la meta, sin comparar alternativas ni acumular información sobre el costo de las rutas exploradas, de modo que la calidad de su solución depende enteramente del orden en que expanda los sucesores. Los datos lo confirman con claridad: en `tinyDiagnostic` obtuvo un costo de 95 frente a los 24 de UCS, es decir, aproximadamente cuatro veces el óptimo, y en `antennaServiceRing` obtuvo 326 frente a 59, cerca de cinco veces y media el óptimo. No existe ninguna configuración del problema que corrija esta limitación, pues es consecuencia directa de su criterio de selección.

BFS es óptimo en el número de movimientos de manera incondicional, ya que explora por niveles y encuentra siempre la solución de menor profundidad. Sin embargo, solo es óptimo en costo total cuando todas las acciones tienen el mismo costo. El contraste entre dos de los mapas evaluados ilustra esta distinción de forma precisa. En `tinyDiagnostic`, BFS y UCS produjeron ambos a un costo de 24, es decir, coincidieron. En `radiationShortcut`, en cambio, BFS obtuvo 84 frente a los 51 de UCS, una diferencia del sesenta y cinco por ciento.

La interpretación de este contraste es el punto central del análisis: la coincidencia

observada en tinyDiagnostic es una propiedad del mapa y no una garantía del algoritmo. En ese mapa el camino de menor número de pasos resultó ser también el de menor costo, porque atraviesa únicamente corredores de costo uniforme. En radiationShortcut esa condición no se cumple, ya que existe una ruta corta que atraviesa zonas de radiación de costo 5 y una ruta más larga que las evita. BFS se comportó exactamente igual en ambos casos, minimizando movimientos; lo que cambió fue la estructura de costos del terreno. Un solo paso por una zona de radiación equivale, en costo, a cinco pasos por corredor normal, de modo que reducir el número de movimientos puede aumentar el costo total.

UCS es óptimo siempre que los costos de acción sean estrictamente positivos, condición que se cumple en todos los mapas del taller. Su optimalidad depende, además, de un detalle de implementación que no puede omitirse: la prueba de meta debe realizarse al extraer el nodo de la cola de prioridad y no al generarlo. Con costos variables como los de este taller, el primer nodo meta generado no es necesariamente el más barato, pues una ruta corta a través de terreno costoso puede generarse antes que una ruta larga a través de terreno económico. La cola de prioridad garantiza que, en el momento de extraer un estado, su costo acumulado g ya es el óptimo para ese estado, y solo entonces es correcto retornar la solución.

A^* es óptimo cuando la heurística es admisible, es decir, cuando nunca sobrestima el costo real restante hasta la meta. En búsqueda en grafo, que es la variante implementada en este taller, se requiere adicionalmente que la heurística sea consistente, ya que de lo contrario un estado podría alcanzarse posteriormente por un camino más económico después de haber sido expandido. Verificamos el comportamiento del algoritmo utilizando nullHeuristic, que es trivialmente admisible y consistente por ser idénticamente cero: sobre radiationShortcut produjo un costo de 51 y 251 nodos expandidos, valores idénticos a los de UCS, lo cual es esperable porque con h igual a cero la prioridad $f = g + h$ se reduce a g y A^* degenera exactamente en UCS.

Como verificación independiente de la optimalidad de UCS puede emplearse la distancia de Manhattan como cota inferior. Dado que el robot solo se desplaza en las cuatro direcciones ortogonales y cada movimiento cuesta como mínimo 1, ningún camino entre dos celdas puede costar menos que la distancia de Manhattan que las separa. En tinyDiagnostic, la distancia entre la posición inicial (2,10) y la terminal (18,2) es de 16 más 8, esto es, 24, y UCS obtuvo un costo de exactamente 24. El algoritmo alcanzó por tanto el mínimo absoluto demostrable, y esta afirmación no depende de confiar en la implementación sino que se sigue de un argumento independiente.

Conviene precisar que esta cota es inferior y no exacta. En radiationShortcut la distancia de Manhattan entre (2,3) y (24,13) es de 32, mientras que UCS obtuvo un costo de 51. La diferencia se explica porque ese mapa no contiene ninguna celda de corredor normal: todo el terreno transitable corresponde a corredores deteriorados, zonas de baja gravedad y zonas de radiación, de manera que el paso más económico disponible cuesta 2 y no 1. La cota inferior efectiva se eleva en consecuencia, y el valor 51 resulta plenamente consistente con esa restricción.

6b. Reflexión sobre el uso de inteligencia artificial

A continuación se detalla el proceso de iteración, depuración y adaptación lógica realizado por el grupo durante el desarrollo de los algoritmos de búsqueda y las heurísticas para la estación espacial Aurora. El registro documenta cómo se integraron herramientas como OpenAI Codex / ChatGPT para estructurar la sintaxis, manteniendo el control lógico y analítico sobre el modelo del problema.

Trazabilidad y Gestión de Prompts por Punto

La siguiente tabla resume las consultas principales, las respuestas generadas por los modelos y la decisión técnica tomada por el equipo frente a cada bloque del taller.

Fase / Algoritmo	Prompt Utilizado	Sugerencia de la IA	Adaptación del Grupo (Aceptado/Rechazado)
Puntos 1 y 2 (DFS / BFS)	"Genera la estructura base para un algoritmo DFS y BFS en Python que retorne una lista de acciones, usando un conjunto de visitados."	Estructura genérica usando <code>list.pop()</code> para DFS y <code>collections.deque</code> para BFS, almacenando solo los nodos visitados.	Adaptado. La IA no incluyó el seguimiento de la ruta (las acciones). Modificamos la pila/cola para almacenar tuplas (<code>estado_actual</code> , <code>ruta_hasta_aqui</code>), acoplándolo a <code>getSuccessors()</code> .
Punto 3 (UCS)	"¿Cómo implemento una PriorityQueue en Python para Uniform Cost Search donde el costo acumulado determine la	Uso de <code>heapq</code> estándar de Python introduciendo tuplas (<code>costo</code> , <code>nodo</code>).	Rechazado parcialmente. Para mantener compatibilidad con <code>algorithms/utils.py</code> , usamos la clase <code>PriorityQueue</code> entregada en el proyecto, pasando el costo <code>g(n)</code> como el parámetro de prioridad en el método <code>push()</code> .

Punto 4 (Módulo Obligatorio)	"Tengo un estado compuesto (position, hasModule). Si el robot recoge el módulo, el costo de las acciones se duplica. ¿Cómo modifico la función de costo?"	La IA sugirió iterar sobre todo el mapa y cambiar la matriz de costos globalmente una vez el módulo es recogido.	Totalmente Rechazado. Modificar el mapa global viola el principio de que el costo depende del *estado*. Implementamos la lógica dentro de getStepCost multiplicando por 2 el costo base solo si hasModule es True.
Punto 5 (Heurística A*)	"Diseña una heurística admisible para A* donde debo recoger un kit K, reparar sistemas T y llegar a C. El estado es (pos, hasKit, pendingSystems)."	Retornar la suma de las distancias de Manhattan desde la posición actual a todos los sistemas T pendientes, más la distancia a C.	Rechazado. Sumar las distancias sobreestima el costo real, haciendo la heurística inadmisibile. Cambiamos la lógica para usar la distancia al sistema más lejano o calcular un árbol de expansión mínima (MST) relajado.

Comprensión Analítica de las Soluciones

A lo largo de la implementación, el desafío principal no fue escribir código Python, sino estructurar matemáticamente el espacio de estados. El enfoque que adoptamos fue muy similar al modelado riguroso de variables de estado: las transiciones y los costos deben definirse de forma directa a través de las condiciones del momento, sin introducir variables globales ni operadores ajenos a la dinámica del sistema.

- **Expansión vs. Generación:** Al depurar con IA, notamos que a menudo confundía el momento adecuado para marcar un nodo como visitado en algoritmos óptimos (UCS y A*). Comprendimos que la comprobación de visitados debe hacerse *al extraer* el nodo de la frontera, no al generarlo, para garantizar la optimalidad.
- **El estado compuesto:** En el problema del módulo (Punto 4), entender que (posicion, hasModule) constituye un estado único nos permitió aislar el comportamiento de los costos. La IA intentaba resolverlo modificando la

infraestructura, pero la interfaz de búsqueda requiere que cada estado contenga toda la información necesaria para derivar sus sucesores de manera independiente.

- **Admisibilidad de la Heurística:** Para `systemRepairHeuristic`, el costo estimado nunca debe superar el costo real. Entendimos que si relajamos el problema (asumiendo que podemos volar entre los sistemas averiados), la distancia máxima o el costo de un MST siempre será menor o igual al recorrido real en la cuadrícula sorteando muros.

Beneficios y Limitaciones del Uso de IA

Beneficios: Las herramientas de IA actuaron como un excelente asistente de sintaxis. Aceleraron la construcción del andamiaje (boilerplate) para extraer elementos de las pilas y colas, y fueron de gran utilidad para recordar convenciones de métodos integrados en Python (como el manejo de tuplas inmutables al definir el conjunto de sistemas pendientes).

Limitaciones y Dificultades: El mayor obstáculo fue la dificultad (casi imposibilidad) de la IA para abstraer reglas de dominio muy específicas sin romper los principios teóricos de la Inteligencia Artificial. Por ejemplo:

1. Cuando se le pedía minimizar el número de pasos (BFS) vs minimizar el costo (UCS), la IA a menudo reciclaba la misma estructura de BFS añadiéndole un contador de costo, ignorando que UCS requiere obligatoriamente una cola de prioridad.
2. La creación del prompt para la heurística fue compleja. Había que explicarle explícitamente a la IA qué era una heurística admisible y consistente, y aun así, sus respuestas matemáticas tendían a la sobreestimación por no comprender que los trayectos entre múltiples puntos pueden superponerse espacialmente.

En conclusión, la iteración con la IA reafirmó que el modelo computacional no reemplaza el criterio de ingeniería. La IA generó propuestas que parecían correctas en código, pero que hubieran resultado en ciclos infinitos o rutas subóptimas si no hubiésemos aplicado nuestro conocimiento teórico sobre espacios de búsqueda para auditar y corregir las matrices y lógicas sugeridas.